

Estrutura de Dados II — AL0507

Lista de Exercícios: Construção de Código | Prova 1 · Turma 70 · 1º Semestre 2026

Linguagem: C ou C++ | Implemente as funções pedidas completando o código fornecido.

1. Pilha com Vetor

Q1. Complete a função **push()** e **pop()** de uma Pilha implementada com vetor. A pilha tem capacidade MAX e usa a variável *topo* para controlar o índice do elemento no topo (-1 significa pilha vazia). **push()** deve retornar 0 em caso de pilha cheia e 1 em sucesso. **pop()** deve retornar -1 se a pilha estiver vazia, ou o valor removido.

■ Dica: lembre de verificar overflow (push) e underflow (pop).

```
#define MAX 100

typedef struct {
    int dados[MAX];
    int topo;
} Pilha;

void inicializar(Pilha *p) {
    p->topo = -1;
}

// Complete abaixo:
int push(Pilha *p, int valor) {
    // sua implementacao aqui
}

int pop(Pilha *p) {
    // sua implementacao aqui
}
```

2. Fila Encadeada

Q2. Implemente as funções **enqueue()** e **dequeue()** para uma Fila com Lista Encadeada. **enqueue()** insere no final e **dequeue()** remove do início, retornando o valor removido ou -1 se a fila estiver vazia.

■ Dica: mantenha ponteiros para o início (frente) e fim (tras) da fila.

```
typedef struct No {
    int valor;
    struct No *prox;
} No;

typedef struct {
    No *frente;
```

```

        No *tras;
    } Fila;

void inicializar(Fila *f) {
    f->frente = NULL;
    f->tras = NULL;
}

// Complete abaixo:
void enqueue(Fila *f, int valor) {
    // sua implementacao aqui
}

int dequeue(Fila *f) {
    // sua implementacao aqui
}

```

3. Busca Binária

Q3. Implemente a função **buscaBinaria()** de forma iterativa. Ela recebe um vetor ordenado v , seu tamanho n e o valor buscado $alvo$. Deve retornar o índice do elemento se encontrado, ou -1 caso contrário.

■ Dica: use as variáveis **inicio**, **fim** e **meio**. Atualize-as a cada iteração.

```

int buscaBinaria(int v[], int n, int alvo) {
    // sua implementacao aqui
}

// Exemplo de uso:
// int v[] = {2, 5, 8, 12, 16, 23, 38, 56};
// buscaBinaria(v, 8, 23) deve retornar 5

```

4. Ordenação — Merge Sort

Q4. Complete a função **merge()** usada pelo Merge Sort. Ela recebe o vetor v e os índices esq , $meio$ e dir , e deve mesclar as duas metades já ordenadas em ordem crescente. A função **mergeSort()** já está implementada para referência.

■ Dica: use um vetor auxiliar temporário para a mesclagem.

```

#include <stdlib.h>
#include <string.h>

// Complete a funcao merge:
void merge(int v[], int esq, int meio, int dir) {
    // sua implementacao aqui
}

// Referencia - nao alterar:
void mergeSort(int v[], int esq, int dir) {
    if (esq < dir) {

```

```

        int meio = (esq + dir) / 2;
        mergeSort(v, esq, meio);
        mergeSort(v, meio + 1, dir);
        merge(v, esq, meio, dir);
    }
}

```

5. Árvore Binária de Busca (ABB)

Q5. Implemente a função **inserir()** em uma ABB. Ela deve inserir o valor respeitando a propriedade da ABB (menores à esquerda, maiores à direita) e retornar o ponteiro para a raiz.

■ Dica: use recursão. Se o nó for NULL, crie um novo nó e retorne-o.

```

#include <stdlib.h>

typedef struct No {
    int valor;
    struct No *esq;
    struct No *dir;
} No;

No* criarNo(int valor) {
    No *novo = (No*) malloc(sizeof(No));
    novo->valor = valor;
    novo->esq = NULL;
    novo->dir = NULL;
    return novo;
}

// Complete abaixo:
No* inserir(No *raiz, int valor) {
    // sua implementacao aqui
}

```

Q6. Implemente o percurso **em-ordem (in-order)** da ABB, imprimindo os valores em ordem crescente.

■ Dica: Esquerda → Raiz → Direita.

```

void emOrdem(No *raiz) {
    // sua implementacao aqui
}

```

Q7. Implemente a função **buscar()** na ABB. Ela deve retornar o ponteiro para o nó com o valor procurado, ou NULL se não encontrado.

■ Dica: compare o valor com a raiz e desça para esquerda ou direita conforme necessário.

```

No* buscar(No *raiz, int valor) {
    // sua implementacao aqui
}

```

Gabarito — Implementações Completas

Q1 — Push e Pop (Pilha com Vetor)

```
int push(Pilha *p, int valor) {
    if (p->topo == MAX - 1) return 0; // pilha cheia
    p->dados[++(p->topo)] = valor;
    return 1;
}

int pop(Pilha *p) {
    if (p->topo == -1) return -1; // pilha vazia
    return p->dados[(p->topo)--];
}
```

Explicacao: push verifica se topo == MAX-1 (overflow), incrementa topo e insere. pop verifica se topo == -1 (underflow), retorna o valor no topo e decrementa.

Q2 — Enqueue e Dequeue (Fila Encadeada)

```
void enqueue(Fila *f, int valor) {
    No *novo = (No*) malloc(sizeof(No));
    novo->valor = valor;
    novo->prox = NULL;
    if (f->tras == NULL) {
        f->frente = novo;
        f->tras = novo;
    } else {
        f->tras->prox = novo;
        f->tras = novo;
    }
}

int dequeue(Fila *f) {
    if (f->frente == NULL) return -1; // fila vazia
    No *temp = f->frente;
    int valor = temp->valor;
    f->frente = f->frente->prox;
    if (f->frente == NULL) f->tras = NULL;
    free(temp);
    return valor;
}
```

Explicacao: enqueue aloca novo no e atualiza o ponteiro tras. dequeue salva o valor do no da frente, avanca o ponteiro frente e libera a memoria.

Q3 — Busca Binaria Iterativa

```
int buscaBinaria(int v[], int n, int alvo) {
```

```

    int inicio = 0, fim = n - 1;
    while (inicio <= fim) {
        int meio = (inicio + fim) / 2;
        if (v[meio] == alvo) return meio;
        else if (v[meio] < alvo) inicio = meio + 1;
        else fim = meio - 1;
    }
    return -1; // nao encontrado
}

```

Explicacao: a cada iteracao, calcula-se o indice do meio. Se o valor for o buscado, retorna o indice. Se o alvo for maior, busca na metade direita; se menor, na metade esquerda. Complexidade: $O(\log n)$.

Q4 — Merge (Merge Sort)

```

void merge(int v[], int esq, int meio, int dir) {
    int n1 = meio - esq + 1;
    int n2 = dir - meio;
    int *L = (int*) malloc(n1 * sizeof(int));
    int *R = (int*) malloc(n2 * sizeof(int));

    for (int i = 0; i < n1; i++) L[i] = v[esq + i];
    for (int j = 0; j < n2; j++) R[j] = v[meio + 1 + j];

    int i = 0, j = 0, k = esq;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) v[k++] = L[i++];
        else v[k++] = R[j++];
    }
    while (i < n1) v[k++] = L[i++];
    while (j < n2) v[k++] = R[j++];

    free(L);
    free(R);
}

```

Explicacao: copia as duas metades para vetores auxiliares L e R, depois mescla de volta para v comparando elemento a elemento. Os elementos restantes sao copiados diretamente.

Q5 — Inserir na ABB

```

No* inserir(No *raiz, int valor) {
    if (raiz == NULL) return criarNo(valor);
    if (valor < raiz->valor)
        raiz->esq = inserir(raiz->esq, valor);
    else if (valor > raiz->valor)
        raiz->dir = inserir(raiz->dir, valor);
    // valor igual: ignora duplicata
    return raiz;
}

```

Explicacao: se a raiz for NULL, cria e retorna novo no. Se o valor for menor, insere recursivamente na subarvore esquerda; se maior, na direita. Duplicatas sao ignoradas nesta implementacao.

Q6 — Percurso Em-Ordem

```
void emOrdem(No *raiz) {
    if (raiz == NULL) return;
    emOrdem(raiz->esq);
    printf("%d ", raiz->valor);
    emOrdem(raiz->dir);
}
```

Explicacao: visita recursivamente a subarvore esquerda, imprime a raiz e depois visita a direita. Em uma ABB, isso garante saida em ordem crescente.

Q7 — Buscar na ABB

```
No* buscar(No *raiz, int valor) {
    if (raiz == NULL) return NULL;
    if (valor == raiz->valor) return raiz;
    if (valor < raiz->valor)
        return buscar(raiz->esq, valor);
    return buscar(raiz->dir, valor);
}
```

Explicacao: se a raiz for NULL, o valor nao existe na arvore. Se o valor for igual ao da raiz, retorna o no. Caso contrario, desce para a subarvore esquerda (menor) ou direita (maior). Complexidade: $O(\log n)$ em arvore balanceada, $O(n)$ no pior caso.